In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from scipy import optimize
from datetime import datetime
import json
```

In [3]:
```
from pathlib import Path

base_dir = Path(r"C:\Users\USER\OneDrive\Documents\HNG_Internship_Git\Stage_4\Stage

paths = {
    "costs": base_dir / "Annual_Cost_Structure_perhectare.xlsx",
    "climate": base_dir / "Climate_Data.xlsx",
    "prices": base_dir / "PalmOil_Prices.xlsx",
    "production": base_dir / "PalmOil_Production.xlsx",
    "states": base_dir / "State_Level_Production.xlsx"
}

out_dir = base_dir / "analysis_outputs"
```

In [4]:
```
# parameters
years_model = [2020, 2021, 2022, 2023, 2024]  # 2020-2024 historical for annual vie
project_life = 20                      # years for NPV/IRR
maturation_years = 3                   # immature years before full yield
discount_rate = 0.12                   # for NPV/annuity
target_profit = 1_000_000_000          # ₦1bn target
selected_states = ['Edo', 'Ondo', 'Cross River']
```

In [5]:
```
# Helper functions

def annualised_capex(capex_per_ha, r=discount_rate, n=project_life):
    # annuity factor
    if capex_per_ha <= 0:
        return 0.0
    af = (r) / (1 - (1 + r) ** -n)
    return capex_per_ha * af

def compute_npv(cashflows, r=discount_rate):
    # cashflows: list or array with cashflow at t=0 (negative) to t=N
    return sum([cf / ((1 + r) ** t) for t, cf in enumerate(cashflows)])

def compute_irr(cashflows):
    try:
        return optimize.newton(lambda rr: compute_npv(cashflows, rr), 0.2)
    except Exception:
        # fallback: use numpy's irr on finite series (requires at least one sign ch
        try:
            return np.irr(cashflows)
        except:
            return np.nan
```

In [ ]:
```
# Load Excel sheets (attempt graceful reading)

def read_sheet(path):
    if not path.exists():
        raise FileNotFoundError(f"File not found: {path}")
    # read first sheet by default
    return pd.read_excel(path, engine="openpyxl")

df_costs = read_sheet(paths["costs"])
df_setup = read_sheet(paths["setup"])
df_states = read_sheet(paths["states"])
df_price = read_sheet(paths["price"])
df_climate = read_sheet(paths["climate"])
df_companies = read_sheet(paths["companies"])
df_national = read_sheet(paths["national"])
df_market = read_sheet(paths["market"])
```

In [ ]:
```
# standardize column names to lower for safer joins
def lc(df):
    df.columns = [c.strip() for c in df.columns]
    return df

for name, df in [
    ("df_states", df_states),
    ("df_costs", df_costs),
    ("df_setup", df_setup),
    ("df_price", df_price),
    ("df_climate", df_climate),
    ("df_companies", df_companies),
    ("df_national", df_national),
    ("df_market", df_market),
]:
    print(name, type(df))
```

In [ ]:
```
# Ensure numeric columns are numeric

for d in [df_states, df_costs, df_setup, df_price, df_climate, df_companies, df_nat
    for col in d.columns:

        if d[col].dtype == object:

            try:
                d[col] = d[col].astype(str).str.replace('[₦$,]', '', regex=True).st
                d[col] = pd.to_numeric(d[col], errors='ignore')
            except Exception:
                pass
```

In [ ]:
```
price_df = df_price.copy()
price_df.columns = [c.lower() for c in price_df.columns]
if 'date' in price_df.columns:
    # extract year from date-like string
    try:
        price_df['year'] = pd.to_datetime(price_df['date'], errors='coerce').dt.yea
    except:
```

```
        price_df['year'] = price_df['date'].astype(str).str.extract(r'(\d{4})').ast
elif 'year' in price_df.columns:
    price_df['year'] = price_df['year']
elif 'csvyear' in price_df.columns:
    price_df['year'] = price_df['csvyear']
else:

    price_map = {}
    for y in years_model:

        price_map[y] = price_map.get(y, np.nan)
    price_df['year'] = price_df.get('year', np.nan)


if 'price_ngn_per_ton' in price_df.columns:
    price_year = price_df.groupby('year')['price_ngn_per_ton'].median().to_dict()
else:

    candidates = [c for c in price_df.columns if 'price' in c]
    if candidates:
        price_year = price_df.groupby('year')[candidates[0]].median().to_dict()
    else:

        if 'year' in df_national.columns and 'cpo_price_ngn_per_ton' in df_national
            price_year = df_national.set_index('year')['cpo_price_ngn_per_ton'].to_
        else:

            price_year = {2020:1150000, 2021: 1300000, 2022:1450000, 2023:1650000,
```

In [ ]:

```
# Fill gaps for the five years of interest
price_series = {y: price_year.get(y, price_year.get(2024, 1850000)) for y in years_

def find_value(df, keywords):
    for k in keywords:
        for col in df.columns:
            if k.lower() in str(col).lower():
                # if column contains values across scenarios (Conservative/Base/Opt
                if 'base' in df.columns.str.lower().tolist():
                    row = df[df[df.columns[0]].astype(str).str.contains(k, case=Fal
                    if not row.empty:
                        # try to find 'Base' like column
                        for c in df.columns:
                            if 'base' in c.lower():
                                return float(row[c].iloc[0])
                # fallback: search row
                row = df[df[df.columns[0]].astype(str).str.contains(k, case=False,
                if not row.empty:
                    # choose the last numeric column
                    numeric_cols = [c for c in df.columns if pd.api.types.is_numeri
                    if numeric_cols:
                        return float(row[numeric_cols[0]].iloc[0])
    return None

base_opex = find_value(df_costs, ['Total_Operating_Cost','Total Operating Cost','To
base_setup = find_value(df_setup, ['Total_Setup_Cost','Total Setup Cost','Total_Set
```

```
if base_opex is None:
    base_opex = 1_100_000
if base_setup is None:
    base_setup = 2_400_000
```

In [ ]:

```
states = df_states.copy()

if 'state' not in states.columns.str.lower().tolist():

    states.rename(columns={states.columns[0]:'State'}, inplace=True)

yield_col = None
for c in states.columns:
    if 'yield' in c.lower():
        yield_col = c
        break
if yield_col is None:
    raise ValueError("Could not find yield column in state-level file. Rename to in

states = states.rename(columns={yield_col:'Yield_CPO_t_per_ha'})

# merge climate rainfall if available (expects 'State' + 'Annual_Rainfall_mm' colum
if 'annual_rainfall_mm' in df_climate.columns.str.lower().tolist():
    # standardize column names
    df_climate.columns = [c if c!='Annual_Rainfall_mm' else 'Annual_Rainfall_mm' fo

    try:
        combined_states = states.merge(df_climate[['State','Annual_Rainfall_mm']],
    except Exception:
        combined_states = states.copy()
else:
    combined_states = states.copy()

# Build per-state per-year model rows (selected states or all states)

selected_states = ['Edo','Ondo','Cross River']   # you can expand this list
rows = []
for _, s in combined_states.iterrows():
    state_name = s['State']
    if state_name not in selected_states:
        continue
    labor_mult = s.get('Labor_Cost_Multiplier', 1.0) if 'Labor_Cost_Multiplier' in
    for year in years_model:
        price = price_series.get(year, price_series[2024])
        yield_cpo = float(s['Yield_CPO_t_per_ha'])
        # revenue
        revenue = yield_cpo * price
        # opex adjust for labor multiplier and inflation (5%/yr)
        opex = base_opex * (1.05 ** (year - years_model[0])) * labor_mult
        # annualised capex (annualised across project life)
        ann_capex = annualised_capex(base_setup, r=discount_rate, n=project_life)
        profit_base = revenue - opex - ann_capex
        rows.append({
            'State': state_name,
            'Year': year,
```

```
            'Yield_CPO_t_per_ha': yield_cpo,
            'Price_N_per_t': price,
            'Revenue_per_ha': revenue,
            'Opex_per_ha': opex,
            'Annualised_Capex_per_ha': ann_capex,
            'Profit_per_ha_base': profit_base
        })

df_model = pd.DataFrame(rows)

states.head()
```

In [ ]:
```
combined_states.to_excel('analysis_outputs/merged_states_dataset.xlsx', index=False
```

In [ ]:
```
# Apply scenarios
# ---------------------------
def add_scenarios(df):
    df = df.copy()
    # Conservative
    df['Profit_per_ha_cons'] = (df['Yield_CPO_t_per_ha'] * 0.7) * (df['Price_N_per_
    # Optimistic
    df['Profit_per_ha_opt'] = (df['Yield_CPO_t_per_ha'] * 1.2) * (df['Price_N_per_t
    # Hectares needed for target profit (only if >0)
    for s in ['base','cons','opt']:
        df[f'Hectares_needed_{s}'] = df.apply(lambda r: (target_profit / r[f'Profit
        df[f'Investment_required_{s}'] = df[f'Hectares_needed_{s}'] * base_setup
        # ROI simple: annual profit / investment (annual profit = target_profit)
        df[f'ROI_{s}'] = (target_profit) / df[f'Investment_required_{s}']
        df[f'Payback_years_{s}'] = df[f'Investment_required_{s}'] / target_profit
    return df

# Ensure base profit column named correctly
# df_model = add_scenarios(df_model)
```

In [ ]:
```
def add_scenarios(df):
    df = df.copy()
    # Conservative
    df['Profit_per_ha_cons'] = (df['Yield_CPO_t_per_ha'] * 0.7) * (df['Price_N_per_
    # Optimistic
    df['Profit_per_ha_opt'] = (df['Yield_CPO_t_per_ha'] * 1.2) * (df['Price_N_per_t
    # Hectares needed for target profit (only if >0)
    for s in ['base','cons','opt']:
        df[f'Hectares_needed_{s}'] = df.apply(lambda r: (target_profit / r[f'Profit
        df[f'Investment_required_{s}'] = df[f'Hectares_needed_{s}'] * base_setup
        # ROI simple: annual profit / investment (annual profit = target_profit)
        df[f'ROI_{s}'] = (target_profit) / df[f'Investment_required_{s}']
        df[f'Payback_years_{s}'] = df[f'Investment_required_{s}'] / target_profit
    return df
```

In [ ]:
```
df_model = add_scenarios(df_model)
```

In [ ]:
```
# Recreate your list of selected states
selected_states = ['Edo', 'Ondo', 'Cross River']
```

```
project_results = []
for state in selected_states:
    # take baseline row for 2024
    row2024 = df_model[(df_model['State']==state) & (df_model['Year']==2024)].iloc[
    for sname in ['base','cons','opt']:
        hectares = row2024[f'Hectares_needed_{sname}']
        if pd.isna(hectares) or hectares<=0:
            continue
        capex_total = base_setup * hectares
        # build yearly cashflows length project_life+1 (t0..tN)
        cashflows = []
        cashflows.append(-capex_total)  # t=0

        immature_fracs = [0.2, 0.5, 0.8][:maturation_years]

        profit_per_ha_full = row2024[f'Profit_per_ha_{sname}']
        # for years 1..project_life
        for yr in range(1, project_life+1):
            if yr <= maturation_years:
                frac = immature_fracs[yr-1] if yr-1 < len(immature_fracs) else imma
                annual_profit = profit_per_ha_full * frac * hectares
            else:
                annual_profit = profit_per_ha_full * hectares
            cashflows.append(annual_profit)
        # compute NPV & IRR
        npv_val = compute_npv(cashflows, discount_rate)
        try:
            irr_val = compute_irr(cashflows)
        except:
            irr_val = np.nan
        project_results.append({
            'State': state,
            'Scenario': sname,
            'Hectares': hectares,
            'CapEx_total_NGN': capex_total,
            'NPV_NGN': npv_val,
            'IRR': irr_val,
            'Payback_simple_years': row2024[f'Payback_years_{sname}'],
            'ROI_simple': row2024[f'ROI_{sname}']
        })

df_projects = pd.DataFrame(project_results)
```

In [ ]:

```
sens = []
row2024s = df_model[df_model['Year']==2024]
for _, r in row2024s.iterrows():
    state = r['State']
    base_hect = r['Hectares_needed_base']
    # price sensitivity
    for change in [-0.2, -0.1, -0.05, 0.05, 0.1, 0.2]:
        price_mod = r['Price_N_per_t'] * (1 + change)
        rev = r['Yield_CPO_t_per_ha'] * price_mod
        profit = rev - r['Opex_per_ha'] - r['Annualised_Capex_per_ha']
        hect_need = (target_profit / profit) if profit>0 else np.nan
        sens.append({'State':state, 'Variable':'Price', 'Change':change, 'Hectares_
    # yield sensitivity
```

```
    for change in [-0.2, -0.1, -0.05, 0.05, 0.1, 0.2]:
        yld_mod = r['Yield_CPO_t_per_ha'] * (1 + change)
        rev = yld_mod * r['Price_N_per_t']
        profit = rev - r['Opex_per_ha'] - r['Annualised_Capex_per_ha']
        hect_need = (target_profit / profit) if profit>0 else np.nan
        sens.append({'State':state, 'Variable':'Yield', 'Change':change, 'Hectares_

df_sens = pd.DataFrame(sens)
```

In [ ]:

```
# Save outputs
df_model.to_csv(out_dir / "palm_profitability_model_2020_2024_raw.csv", index=False
df_projects.to_csv(out_dir / "palm_project_npv_irr_2024.csv", index=False)
df_sens.to_csv(out_dir / "palm_sensitivity_2024.csv", index=False)

# Save cleaned states used
combined_states[combined_states['State'].isin(selected_states)].to_csv(out_dir / "s
```